

Övning 7: Cars – Full CRUD i Frontend

Idag ska ni koppla ihop ert nyskapade C# Minimal API med en frontend i HTML och JavaScript. Ni har sett hur vi hämtar och visar alla bilar (**READ**). Nu är det er tur att bygga klart resten av applikationen så att vi kan lägga till, uppdatera och ta bort bilar direkt från webbläsaren!

Mål med övningen

- Förstå hur asynkron programmering med `async`, `await` och `fetch` fungerar i praktiken.
- Kunna skicka data till ett API med olika HTTP-metoder (GET, POST, PUT, DELETE).
- Lära sig att tolka och arbeta mot ett API:s dokumentation (Swagger).

Förberedelser & Startskal

1. Säkerställ att ditt backend (Minimal API) körs i terminalen (`dotnet run`) och att du kan surfa in på Swagger: <http://localhost:5000/swagger> (eller den port din terminal visar). Se dokumentet **Starta MinimalAPI på 1 minut**
2. Använd **övning7.html** för att få en startup-fil för frontend.
3. Öppna **övning7.html** med **Live Server** i VS Code.

Din Utmaning

Uppgiften är uppdelad i tre nivåer.

Nivå 1: Skapa ny bil (CREATE - POST)

När vi skickar data till servern för att spara en ny bil måste vi berätta för API:et att vi skickar JSON, och vi måste paketera om vårt JavaScript-objekt till en textsträng.

- **HTTP-metod:** POST
- **Endpoint:** `/api/bilar`
- **Dina uppgifter:**
 - Lyssna på formulärets `submit`-event (kom ihåg `event.preventDefault()`).

- Hämta värdena från input-fälten. **Viktigt:** Årsmodellen (year) måste konverteras till en siffra (int) med `parseInt()` innan den skickas, annars kommer ditt C#-API att neka anropet!
- Gör ett fetch-anrop där du skickar med method: 'POST', sätter rätt headers (så att API:et vet att det är JSON) samt skickar objektet i body (använd `JSON.stringify()`).
- Om anropet lyckas (`response.ok`): Töm formuläret och ladda om listan med bilar automatiskt.

Nivå 2: Ta bort en bil (DELETE - DELETE)

För att kunna radera en bil måste vi berätta för API:et *vilken* bil det gäller genom att skicka med bilens unika Id i webbadressen (URL:en).

- **HTTP-metod:** DELETE
- **Endpoint:** `/api/bilar/{id}` (t.ex. `/api/bilar/3`)
- **Dina uppgifter:**
 - När du loopar ut bilarna i din GET-funktion, se till att varje bilkort får en "Ta bort"-knapp.
 - Knappen måste på något sätt veta vilket Id den tillhör. (Tips: Du kan skicka med id:t direkt i ett funktionsanrop på knappen: `onClick="deleteCar(${car.id})"`).
 - Skriv en asynkron funktion `deleteCar(id)` som gör ett fetch-anrop till rätt URL med method: 'DELETE'.
 - Om raderingen lyckas på servern: Ladda om bilistan så att den raderade bilen försvinner från skärmen.

Nivå 3: Redigera en bil (UPDATE - PUT)

Att uppdatera en bil är i särklass svårast eftersom det kräver ett flöde i två steg. Detta är en frivillig utmaning för dig som vill ha en rejäl nöt att knäcka!

- **HTTP-metod:** PUT
- **Endpoint:** `/api/bilar/{id}`
- **Dina uppgifter:**
 - Lägg till en "Redigera"-knapp på varje bilkort.
 - När användaren klickar på "Redigera" ska bilens nuvarande uppgifter (brand, model, year, color) hämtas och fyllas i automatiskt i formuläret längst upp på sidan.

- Du måste spara undan bilens Id i JavaScript (t.ex. i en global variabel `let currentEditingId = null;`) så att du vet vilken bil du håller på att ändra.
- När användaren sedan klickar på "Spara bil"-knappen måste din kod kontrollera: *Håller vi på att skapa en ny bil, eller uppdaterar vi en befintlig?*
- Om vi uppdaterar: Gör ett fetch-anrop med method: 'PUT' till `/api/bilar/ID` med den nya datan i din body.

Felsökningstips under eftermiddagen:

- **Öppna F12 (Utvecklarverktyg):** Ha alltid fliken **Console** och fliken **Network** öppna. Om något inte fungerar, titta där först!
- **Stavar du rätt?** Kom ihåg att i C# skriver vi Brand (PascalCase), men när datan skickas till JavaScript blir det brand (camelCase). Om du skriver `car.Brand` i JavaScript kommer det att stå `undefined`.
- **Vad säger Swagger?** Om du får felkod 400 Bad Request eller 404 Not Found – gå till Swagger, titta på dokumentationen för den endpoint du anropar och kontrollera att din URL och ditt JSON-objekt ser exakt ut som API:et förväntar sig.